Performance-wise, since creating a k-d tree for the centroids will require `O(k*log(k))` steps, each of which can require up to `O(d)` operations (because we are dealing with `d`-dimensional points), then the whole classification step will require `O(d*k*log(k) + n*2d + n*d*log(k))= O(n*2d + d*(n+k)*log(k))` steps, instead of `O(n*k*d)` operations.

Formally, we can work out the exact conditions for which `O(n*2d + d*(n+k)*log(k)) < O(n*k*d)`; for our purpose, however, we can just informally use some intuition to see that:

- `n*2d < n*k*d` $\iff$ `d << k`
- `d*(n+k)*log(k) < n*k*d` $\iff$ `(n+k)*log(k) < n*k` $\iff$ `n < n*k/log(k)–k` This can be shown to hold for `n > k`, but plotting the difference between the two sides shows that, for a fixed `n`, the difference grows as `k` grows: so the saving is more noticeable when there are many centroids.

The whole algorithm, assuming that clusters are evenly distributed with approximately `n/k` points each, then would run in `O(m*(n*2d + d*(n+k)*log(k) + n*d)) = O(m*(n*2d + d*(n+k)*log(k)))` and so, in theory, and net of the constant multipliers and implementation details, it seems that it could be a good idea to use a data structure like k-d trees to improve nearest neighbor search.

As we have seen, however, if the theoretical margin is small, sometimes a more complex implementation only beats asymptotically worse solutions for large, or at times very large, inputs. To double check if in practice it is worth to go